

UNIVERSIDAD

FACULTAD DE INGENIERÍA

Carrera de Ingeniería de Sistemas y Software

INFORME FINAL DE PROYECTO

TASK MANAGER

Sistema de Gestión de Tareas con Spring Boot, Spring Security y JWT

CURSO

Marcos de Desarrollo Web

DOCENTE

Paul Alberto Vizcarra Rodriguez

GRUPO N.º 10

INTEGRANTES

Morales Bueno, Herbert

Espinoza Navarro, Leyla

Garcia Huaman, Jose Gabriel

Quispe Gamarra, Luis Alberto

Lima – Perú

2026

Índice

Fuentes de Información.....	3
2. Configuración del Sistema.....	4
2.1 Dependencias declaradas en pom.xml.....	4
2.2 application.properties — explicación línea por línea.....	4
2.3 Variables de entorno del proyecto.....	4
3. Base de Datos y Modelado.....	7
3.1 Script de inicio: creación de tablas (schema.sql).....	7
3.2 Script de poblamiento de datos (data.sql).....	7
3.3 Modelo Entidad-Relación.....	7
3.4 Entidades JPA y mapeo de relaciones.....	7
4. Funcionalidades e Interfaz.....	10
4.1 Operaciones CRUD.....	10
4.2 Panel de Administración.....	10
4.3 Diseño de la interfaz unificada.....	10
5. Seguridad.....	12
5.1 Encriptación de contraseñas.....	12
5.2 Arquitectura JWT.....	12
5.3 Autorización y gestión de roles.....	12
5.4 CSRF, cookies y otras decisiones de seguridad.....	12
6. Despliegue.....	15
6.1 Requisitos previos.....	15
6.2 – 6.7 Pasos de instalación y verificación.....	15

Fuentes de Información

El presente informe fue elaborado de forma íntegra y original por los integrantes del grupo, a partir del análisis directo del código fuente del proyecto Task Manager (repositorio propio del equipo) y de la documentación oficial de las tecnologías empleadas. No se ha copiado contenido de terceros; las referencias técnicas consultadas como apoyo conceptual fueron las siguientes:

- Spring Framework / Spring Boot — Documentación oficial: <https://docs.spring.io/spring-boot/>
- Spring Security — Documentación oficial: <https://docs.spring.io/spring-security/reference/>
- JWT (Java JWT) — Documentación de la librería: <https://github.com/jwt/jwt>
- Thymeleaf — Documentación oficial: <https://www.thymeleaf.org/documentation.html>
- MySQL 8 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- Bootstrap 5.3 — Documentación oficial: <https://getbootstrap.com/docs/5.3/>

Todo el código, los scripts SQL y los diagramas presentados en este documento corresponden al proyecto desarrollado por el grupo, alojado en la carpeta del proyecto "task-manager" (paquete base com.prymevolution.taskmanager).

2. Configuración del Sistema

El proyecto se gestiona con Apache Maven y se apoya en dos archivos de configuración: `pom.xml`, que declara las dependencias del proyecto, y `application.properties`, que centraliza toda la configuración específica del entorno de ejecución (puerto del servidor, conexión a la base de datos, comportamiento de Hibernate/JPA, Thymeleaf y los parámetros del token JWT). A continuación se explica exactamente qué contiene cada bloque de `application.properties` en este proyecto, no una descripción genérica de Spring Boot.

2.1 Dependencias declaradas en `pom.xml`

El proyecto usa como padre `spring-boot-starter-parent` versión 3.3.5 y fija Java 17 como versión de compilación. Las dependencias relevantes son:

- `spring-boot-starter-web` y `spring-boot-starter-thymeleaf`: exponen el servidor Tomcat embebido y el motor de plantillas server-side.
- `thymeleaf-extras-springsecurity6`: permite usar las etiquetas `sec:authorize` y `sec:authentication` dentro de las vistas HTML (`navbar.html`, `index.html`) para mostrar contenido según el rol o estado de autenticación.
- `spring-boot-starter-data-jpa` + `mysql-connector-j`: habilitan Spring Data JPA sobre MySQL 8.
- `spring-boot-starter-security`: habilita el módulo de seguridad de Spring.
- `io.jsonwebtoken`: `jjwt-api`, `jjwt-impl` y `jjwt-jackson` (versión 0.12.6): librería JWT usada para firmar, generar y verificar los tokens JWT en `JwtUtil`.
- `spring-boot-starter-validation`: habilita las anotaciones de Jakarta Bean Validation (`@NotBlank`, `@Size`, `@Email`, `@FutureOrPresent`) usadas en las entidades `User` y `Task`.
- `lombok`: genera en tiempo de compilación los getters, setters, constructores y builders de entidades, DTOs y servicios, evitando código repetitivo.
- `spring-boot-devtools`: reinicio automático del servidor durante el desarrollo (scope `runtime`, optional).

2.2 `application.properties` — explicación línea por línea

Este archivo está dividido en cinco bloques temáticos. Se transcribe el contenido real del proyecto y se explica el propósito concreto de cada propiedad:

a) Configuración general

```
spring.application.name=task-manager
server.port=8081
```

El servidor embebido Tomcat escucha en el puerto 8081 (no el 8080 por defecto), para evitar conflictos con otros servicios locales durante el desarrollo del equipo.

b) Conexión a MySQL — credenciales por variables de entorno

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/taskmanager_db?
createDatabaseIfNotExist=true&useSSL=false&serverTimezone=America/
Lima&allowPublicKeyRetrieval=true}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Ninguna credencial de base de datos está escrita en texto plano dentro del proyecto. La sintaxis `{VARIABLE:valor_por_defecto}` de Spring Boot primero busca la variable de entorno indicada (`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`); si no existe, usa el valor tras los dos puntos, pensado únicamente para que el proyecto arranque "out of the box" en un entorno local (MySQL en localhost, usuario root sin contraseña, típico de XAMPP/WAMP). En un despliegue real, el equipo exporta estas tres variables en el sistema operativo o en la configuración de ejecución del IDE, de modo que la contraseña real jamás se sube al repositorio. La URL JDBC además incluye `createDatabaseIfNotExist=true` (crea la base `taskmanager_db` si no existe), `useSSL=false` (entorno de desarrollo sin TLS en MySQL), `serverTimezone=America/Lima` (evita desfases de fecha/hora con el huso horario de Perú) y `allowPublicKeyRetrieval=true` (necesario para el plugin de autenticación por defecto de MySQL 8).

c) JPA / Hibernate

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true
```

ddl-auto=update le indica a Hibernate que compare las entidades Java (User, Task) contra las tablas existentes en MySQL y aplique automáticamente los cambios de esquema que falten, sin borrar datos existentes; en este proyecto coexiste con los scripts manuales schema.sql/data.sql como vía alternativa y reproducible. show-sql=false evita imprimir cada sentencia SQL en consola (se activaría en true solo para depuración). El dialecto MySQL8Dialect asegura que Hibernate traduzca los tipos de datos y las funciones al dialecto correcto de MySQL 8.

d) Thymeleaf

```
spring.thymeleaf.cache=false
spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.mode=HTML
```

El caché de plantillas se desactiva (cache=false) para que, durante el desarrollo, los cambios en los archivos .html de templates/ se reflejen de inmediato sin reiniciar el servidor. La codificación UTF-8 garantiza que los acentos y símbolos del español (ñ, tildes) se muestren correctamente en las vistas.

e) JWT (JSON Web Token) — el bloque más importante de la configuración

```
jwt.secret=$
{JWT_SECRET:c2VjcmV0LWtleS1wYXJhLXRhc2stbWFuYWdlci1wcnltZXZvbHV0aW9uLWNhbWJp
YXItZW4tcHJvZHVjY2lvdjg==}
```

```
jwt.expiration=${JWT_EXPIRATION:3600000}
```

jwt.secret es la clave secreta Base64 con la que JwtUtil firma y verifica cada token usando el algoritmo simétrico HMAC-SHA256 (HS256). Al igual que las credenciales de base de datos, se lee de la variable de entorno JWT_SECRET; el valor tras los dos puntos es una clave de desarrollo (decodificada en Base64 equivale a la frase "secret-key-para-task-manager-primevolution-cambiar-en-produccion") que únicamente permite que el proyecto arranque sin configuración adicional en el entorno del equipo. En un despliegue real, el instructivo del proyecto exige sobrescribir JWT_SECRET con una clave aleatoria de al menos 256 bits generada específicamente para producción, distinta a la de desarrollo, para que nadie pueda falsificar tokens conociendo el valor publicado en el repositorio. jwt.expiration=3600000 fija el tiempo de vida del token en 1 hora (3 600 000 milisegundos); pasado ese tiempo, JwtUtil.isValid() lo considera expirado y el usuario debe volver a autenticarse.

Ambas variables (jwt.secret y jwt.expiration) se inyectan directamente en el constructor de JwtUtil mediante @Value("\${jwt.secret}") y @Value("\${jwt.expiration}"), por lo que ese componente es el único lugar del código que conoce la clave secreta real.

2.3 Variables de entorno del proyecto

Variable	Descripción	Valor por defecto
DB_URL	URL JDBC de conexión a MySQL	jdbc:mysql://localhost:3306/taskmanager_db?...
DB_USERNAME	Usuario de la base de datos	root
DB_PASSWORD	Contraseña de la base de datos	(vacío)
JWT_SECRET	Clave Base64 para firmar los JWT (HS256, ≥256 bits)	clave de desarrollo incluida — se cambia en producción
JWT_EXPIRATION	Tiempo de vida del token en milisegundos	3600000 (1 hora)

3. Base de Datos y Modelado

La persistencia del sistema se apoya en MySQL 8 y en Spring Data JPA/Hibernate como capa de mapeo objeto-relacional. El proyecto incluye, dentro de la carpeta database/, dos scripts SQL separados y explícitos: uno de creación de estructura (schema.sql) y otro de poblamiento de datos iniciales (data.sql), tal como exige la rúbrica del curso.

3.1 Script de inicio: creación de la base de datos y tablas (schema.sql)

Este script crea la base de datos taskmanager_db (si no existe) con codificación utf8mb4 y define las dos tablas del sistema, reflejando exactamente el esquema que generan las entidades JPA. Es idempotente: usa CREATE TABLE IF NOT EXISTS, por lo que puede ejecutarse varias veces sin duplicar estructuras.

```
CREATE DATABASE IF NOT EXISTS taskmanager_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE taskmanager_db;

CREATE TABLE IF NOT EXISTS users (
  id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(255) NOT NULL,
  email   VARCHAR(255) NOT NULL,
  role    VARCHAR(20) NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_users_username UNIQUE (username),
  CONSTRAINT uq_users_email   UNIQUE (email)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;

CREATE TABLE IF NOT EXISTS tasks (
  id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  title   VARCHAR(255) NOT NULL,
  description VARCHAR(500),
  due_date DATE NOT NULL,
  priority VARCHAR(20),
  status   VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
  user_id  BIGINT NOT NULL,
  created_at DATETIME,
  CONSTRAINT fk_tasks_user FOREIGN KEY (user_id)
    REFERENCES users (id) ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

Sobre las restricciones aplicadas: username y email tienen restricción UNIQUE para impedir duplicados a nivel de motor de base de datos (además de la validación de aplicación en UserService); la clave foránea fk_tasks_user enlaza tasks.user_id con users.id con ON DELETE CASCADE, de modo que eliminar un usuario elimina automáticamente todas sus tareas sin dejar registros huérfanos; y status tiene un valor por defecto ('PENDIENTE') que coincide con el valor por defecto definido en la entidad Java Task.

3.2 Script de poblamiento de datos (data.sql)

Este segundo script, ejecutado después de schema.sql, inserta las dos cuentas de prueba exigidas por la rúbrica del curso. Usa INSERT IGNORE para ser igualmente idempotente: si las cuentas ya existen (por ejemplo porque DataInitializer ya las sembró en un arranque previo de la aplicación), la sentencia no falla ni duplica registros.

```
USE taskmanager_db;

INSERT IGNORE INTO users (username, password, email, role, enabled) VALUES
('admin', '$2b$10$g3IfVhQwWTDafKG5TLWEu...', 'admin@taskmanager.com', 'ADMIN',
TRUE),
('usuario', '$2b$10$WKkbDcGK70hn.XTKwotVR...', 'usuario@taskmanager.com', 'USER',
TRUE);
```

Las contraseñas se insertan ya como hash BCrypt (factor de coste 10), generado con el mismo algoritmo que usa BCryptPasswordEncoder dentro de la aplicación — nunca se guarda una contraseña en texto plano, ni siquiera en los datos de prueba. Como alternativa a este script, el proyecto incluye el componente DataInitializer (implementa CommandLineRunner), que siembra automáticamente estas mismas dos cuentas más doce tareas de demostración (seis por usuario, repartidas entre los tres estados y las tres prioridades) la primera vez que la aplicación arranca con la tabla users vacía; ambas vías son coherentes entre sí.

3.3 Modelo Entidad-Relación

El modelo de datos del sistema está compuesto por dos tablas relacionadas mediante una relación Uno a Muchos (1:N): un usuario (users) puede tener muchas tareas (tasks), pero cada tarea pertenece exactamente a un usuario. La clave foránea user_id vive en la tabla tasks (el lado "muchos" de la relación) y apunta a users.id, con la política ON DELETE CASCADE ya descrita.

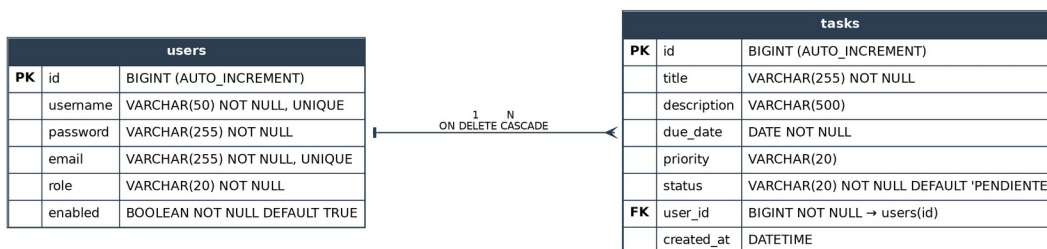


Figura 1. Diagrama Entidad-Relación de la base de datos taskmanager_db.

3.4 Entidades JPA y mapeo de relaciones

Entidad User (tabla users)

Mapeada con @Entity y @Table(name = "users") — el nombre se declara explícitamente porque "user" es palabra reservada en MySQL. Usa las anotaciones de Lombok @Data, @NoArgsConstructor y @AllArgsConstructor para generar automáticamente getters, setters, constructores, equals() y

hashCode()). Sus campos son: id (Long, @Id @GeneratedValue(IDENTITY)), username (@NotBlank, @Size(3,50), único), password (@NotBlank, se guarda como hash BCrypt), email (@NotBlank, @Email, único), role (enum UserRole mapeado con @Enumerated(EnumType.STRING), por defecto USER) y enabled (boolean, por defecto true, permite suspender una cuenta sin borrar su historial).

El lado inverso de la relación se declara como: @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY) private List<Task> tasks;. mappedBy = "user" indica que la clave foránea vive en la entidad Task, no en User, evitando que Hibernate cree una tabla intermedia innecesaria. cascade = CascadeType.ALL propaga a las tareas cualquier operación realizada sobre el usuario — en particular, elimina en cascada todas sus tareas si el usuario es eliminado. fetch = FetchType.LAZY evita cargar la lista completa de tareas cada vez que se consulta un usuario, cargándola solo cuando el código accede explícitamente a user.getTasks().

Entidad Task (tabla tasks)

Mapeada con @Entity y @Table(name = "tasks"). Sus campos son: id (Long, autogenerado), title (@NotBlank, @Size(max=120)), description (@Size(max=500), opcional), dueDate (LocalDate, @NotNull, @FutureOrPresent — no admite fechas pasadas), priority (enum TaskPriority: ALTA/MEDIA/BAJA, @Enumerated(STRING)), status (enum TaskStatus: PENDIENTE/EN_PROGRESO/COMPLETADA, @Enumerated(STRING), valor por defecto PENDIENTE) y createdAt (LocalDateTime, @Column(updatable = false), fecha de auditoría inmutable tras el primer guardado).

El lado propietario de la relación se declara como: @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "user_id", nullable = false) private User user;. Esta anotación es la que efectivamente genera la columna de clave foránea user_id en la tabla tasks; nullable = false garantiza que toda tarea tenga siempre un dueño (no se permiten tareas huérfanas). El método de ciclo de vida @PrePersist establece automáticamente createdAt = LocalDateTime.now() justo antes del INSERT, y refuerza que status nunca sea nulo.

Tanto en User.role como en Task.priority y Task.status se usa EnumType.STRING (guarda el texto "ADMIN", "ALTA", "PENDIENTE", etc.) en lugar de EnumType.ORDINAL (guardaría un índice numérico 0,1,2). Esta decisión de diseño hace que los datos existentes en la base no se corrompan si en el futuro se reordenan o insertan nuevos valores dentro de un enum.

4. Funcionalidades e Interfaz

El sistema implementa un CRUD completo de tareas sobre una arquitectura en capas (Controller → Service → Repository → Base de Datos), con dos experiencias diferenciadas por rol: un panel personal para usuarios USER y un panel global de administración para ADMIN. Toda la interfaz se construye con Thymeleaf (renderizado del lado del servidor) y Bootstrap 5.3.3, con Font Awesome para iconografía, sobre una hoja de estilos propia (styles.css, 233 líneas) que define variables de color corporativas.

4.1 Operaciones CRUD

Operación	Ruta	Comportamiento
Crear	GET/POST /tasks/nueva	Formulario con título, descripción, fecha de vencimiento y prioridad. Aplica validación de Bean Validation + TaskValidator y asigna automáticamente al usuario logueado como dueño.
Leer	GET /tasks	Lista únicamente las tareas del usuario autenticado (findByUserOrderByCreatedAtDesc), ordenadas de la más reciente a la más antigua, con tarjetas resumen (Total, Pendientes, En Progreso, Completadas).
Actualizar	GET/POST /tasks/editar/{id}	Precarga el formulario con los datos actuales; verifica que la tarea pertenezca al usuario antes de permitir la edición; permite además cambiar el estado (campo exclusivo del modo edición).
Eliminar	POST /tasks/eliminar/{id}	Verifica propiedad del recurso y elimina tras confirmación mediante un modal de Bootstrap (evita borrados accidentales).

El controlador TaskController sigue el patrón Post-Redirect-Get (PRG): tras un POST exitoso siempre responde con un redirect (redirect:/tasks) en lugar de renderizar directamente la vista, lo que evita que un F5 del navegador reenvíe el formulario. Los mensajes de confirmación ("Tarea registrada exitosamente", "Tarea actualizada exitosamente", "Tarea eliminada correctamente") viajan como flash attributes, que sobreviven al redirect y se muestran una única vez.

Cada operación de edición y eliminación pasa además por el método privado findOwned(), que constituye una verificación de propiedad a nivel de recurso (object-level authorization): comprueba que la tarea exista y que su usuario dueño coincida con el usuario autenticado, de forma que ningún USER pueda modificar o borrar tareas ajenas aunque conozca su identificador numérico.

4.2 Panel de Administración

Accesible únicamente en /admin para usuarios con rol ADMIN. Muestra métricas globales del sistema (total de usuarios, total de tareas, y conteo de tareas por estado sin filtrar por usuario) y dos tablas: una con todos los usuarios registrados (con opción de eliminarlos, salvo la cuenta admin, protegida en el propio template) y otra con todas las tareas del sistema de todos los usuarios, también eliminables desde ahí. Es el único punto del sistema con visibilidad total sobre los datos de todos los colaboradores.

4.3 Diseño de la interfaz unificada

Todas las vistas comparten una estructura común mediante fragmentos de Thymeleaf reutilizables (th:replace), ubicados en templates/fragments/: head.html (metadatos, título dinámico por página, enlaces a Bootstrap/Font Awesome/CSS propio), navbar.html (barra de navegación que cambia dinámicamente su contenido según el estado de autenticación y el rol, usando sec:authorize), alerts.html (bloque reutilizable de alertas de éxito/error) y footer.html. Esta reutilización evita duplicar HTML entre las más de diez páginas del sistema y asegura una identidad visual consistente.

Las páginas principales son: index.html (landing pública con llamados a la acción para visitantes anónimos, y resumen de estadísticas personales para usuarios autenticados — misma URL, contenido condicional); auth/login.html (formulario con toggle de visibilidad de contraseña en JavaScript vanilla y credenciales de prueba visibles para fines académicos); auth/register.html (formulario de registro con validación visual inline mediante is-invalid/invalid-feedback de Bootstrap); tasks/list.html (tabla de tareas con tarjetas resumen, badges de color por prioridad y estado, y modal de confirmación de borrado); tasks/form.html (formulario unificado que sirve tanto para crear como para editar, controlado por la bandera isEdit, con contador de caracteres en vivo para la descripción); admin/dashboard.html (tablas duales de usuarios y tareas con métricas globales); y error/403.html (página de acceso denegado).

El diseño es completamente responsivo gracias al sistema de grillas de Bootstrap 5 (clases col-6, col-lg-3, etc.), adaptándose sin cambios de código a dispositivos de escritorio, tablets y móviles. Los badges de prioridad (badge-alta en rojo, badge-media en naranja, badge-baja en gris) y de estado (Pendiente en amarillo, En Progreso en celeste, Completada en verde) se generan dinámicamente en el HTML a partir del enum correspondiente, por ejemplo: `th:class="'badge badge-' + ${t.priority.name().toLowerCase()}"`.

5. Seguridad

La protección de la aplicación se apoya íntegramente en Spring Security 6 combinado con una implementación propia de autenticación sin estado (stateless) basada en JSON Web Tokens (JWT), en lugar del esquema clásico de sesión HTTP (HttpSession). Esta decisión de arquitectura está documentada y justificada en el propio proyecto: el servidor nunca almacena estado de sesión; toda petición se autentica desde cero a partir del token JWT que la acompaña.

5.1 Encriptación de contraseñas

Todas las contraseñas se almacenan como hash BCrypt (factor de coste 10), nunca en texto plano. El bean BCryptPasswordEncoder se define en SecurityConfig y se inyecta tanto en UserService.register() (registro por formulario web) como en DataInitializer (cuentas de demostración). BCrypt es un algoritmo de hash adaptativo con salt aleatorio incorporado: dos usuarios con la misma contraseña generan hashes completamente distintos, y el costo computacional del algoritmo hace impracticables los ataques de fuerza bruta.

5.2 Arquitectura JWT — cómo se genera, transporta y valida el token

Un JWT consta de tres partes separadas por puntos — Header.Payload.Signature. El Header identifica el algoritmo de firma (HS256); el Payload contiene los claims: subject (username), un claim personalizado "roles" con las autoridades del usuario, iat (fecha de emisión) y exp (fecha de expiración); la Signature es un HMAC-SHA256 del header y el payload, calculado con la clave secreta jwt.secret, y es lo que garantiza que el contenido no pueda alterarse sin invalidar la firma.

Componentes de la capa de seguridad

Componente	Responsabilidad
JwtUtil	Único componente que conoce la clave secreta. Genera el token (generateToken) firmándolo con Keys.hmacShaKeyFor() y lo valida (isTokenValid) verificando firma, username y expiración.
JwtAuthenticationFilter	Filtro OncePerRequestFilter registrado antes del filtro estándar de usuario/contraseña. Se ejecuta en cada petición: extrae el token de la cabecera Authorization: Bearer <token> o, si no está, de la cookie jwt; si es válido, reconstruye el SecurityContextHolder con un UsernamePasswordAuthenticationToken.
UserDetailsServiceImpl	Puente entre la tabla users y Spring Security: convierte la entidad User en un UserDetails con el prefijo ROLE_ requerido por hasRole(), y marca la cuenta como bloqueada/deshabilitada si enabled = false.
AuthService	Punto único de autenticación: usa AuthenticationManager (con DaoAuthenticationProvider + BCrypt) para validar

Componente	Responsabilidad
	credenciales, y emite el JWT con JwtUtil. Lo reutilizan tanto el login web como el login de la API.
RestAwareAuthenticationEntryPoint	Decide qué responder ante una petición sin autenticar: 401 en JSON si la ruta empieza con /api/**, o redirect a /login si es una petición de navegador.
RestAwareAccessDeniedHandler	Decide qué responder ante un usuario autenticado sin el rol requerido: 403 en JSON para /api/**, o redirect a /error/403 para el navegador.
SecurityConfig	Ensambla toda la cadena de filtros: SessionCreationPolicy.STATELESS, CSRF deshabilitado, reglas de autorización por ruta, registro del filtro JWT antes de UsernamePasswordAuthenticationFilter, y @EnableMethodSecurity para habilitar @PreAuthorize.

Dos vías de autenticación, un único servicio

El sistema ofrece dos formas de obtener el token, ambas delegando en AuthService para no duplicar lógica: AuthRestController (POST /api/auth/login) es una API REST pura, pensada para Postman o un cliente externo — recibe { username, password } en JSON y devuelve { token, username, roles } en el cuerpo de la respuesta. AuthController (POST /login), pensado para el navegador, ejecuta la misma autenticación pero en lugar de devolver el token en el cuerpo, lo guarda en una cookie llamada jwt con los atributos HttpOnly (JavaScript no puede leerla, mitigando robo por XSS), SameSite=Lax (mitiga CSRF) y una vigencia de una hora; el navegador la reenvía automáticamente en cada petición posterior, permitiendo que las vistas Thymeleaf funcionen exactamente igual que con el login por sesión tradicional, sin que el usuario final note ninguna diferencia.

El cierre de sesión (POST /logout) no invalida nada en el servidor —no existe una HttpSession que destruir —: simplemente sobrescribe la cookie jwt con un valor vacío y Max-Age=0, de modo que el navegador deja de enviarla.

5.3 Autorización y gestión de roles

El sistema define dos roles mediante el enum UserRole: USER (acceso exclusivo a sus propias tareas) y ADMIN (visibilidad y gestión global de usuarios y tareas). Ningún usuario puede autoasignarse el rol ADMIN desde el formulario público de registro: UserService.register() fuerza explícitamente user.setRole(UserRole.USER) sin importar qué haya llegado en la petición HTTP, como protección contra ataques de manipulación de parámetros (mass assignment). La única cuenta ADMIN del sistema se crea desde DataInitializer / data.sql.

La autorización se aplica en tres niveles independientes (defensa en profundidad):

- Nivel 1 — Reglas HTTP en SecurityConfig: .requestMatchers("/admin/**").hasRole("ADMIN") exige la autoridad ROLE_ADMIN para cualquier ruta de administración;

`.requestMatchers("/tasks/**").authenticated()` exige solamente estar autenticado (USER o ADMIN); las rutas `/`, `/login`, `/register`, `/css/**`, `/js/**`, `/error` y `/api/auth/**` son públicas (`permitAll()`).

- Nivel 2 — Seguridad a nivel de método: `AdminController` está anotado a nivel de clase con `@PreAuthorize("hasRole('ADMIN')")`, habilitado por `@EnableMethodSecurity` en `SecurityConfig`. Spring evalúa esta expresión SpEL antes de ejecutar cualquier método del controlador, incluso si por algún motivo la regla HTTP no lo hubiera bloqueado.
- Nivel 3 — Verificación de propiedad del recurso: en `TaskController`, el método `findOwned()` comprueba en código que la tarea solicitada pertenezca al usuario autenticado antes de permitir editarla o eliminarla. Spring Security no puede expresar esta regla declarativamente porque depende de un dato de negocio (el dueño de cada fila), así que se implementa explícitamente en el controlador.

5.4 CSRF, cookies y otras decisiones de seguridad

La protección CSRF integrada de Spring Security está deshabilitada (`.csrf(csrf -> csrf.disable())`) de forma deliberada y justificada: dicha protección existe para defender aplicaciones que autentican por cookie de sesión, que es precisamente el vector que este proyecto no usa (`SessionCreationPolicy.STATELESS`). La cookie jwt que sí transporta el token está protegida de forma independiente con los atributos `HttpOnly` y `SameSite=Lax`, que mitigan robo por XSS y reenvío cross-site respectivamente. El atributo `Secure` de la cookie está desactivado únicamente porque el entorno de desarrollo corre sobre HTTP; el propio código señala que debe activarse (`.secure(true)`) en un despliegue real sobre HTTPS.

Adicionalmente, `JwtAuthenticationFilter` nunca deja pasar un token corrupto o expirado silenciosamente: cualquier `JwtException` al parsear el token limpia el `SecurityContextHolder`, y es la combinación de `SecurityConfig` + los handlers `RestAware...` la que decide la respuesta final (401 o redirect a `/login` según el tipo de cliente).

6. Despliegue

El proyecto está preparado para ejecutarse tanto en un entorno de desarrollo local como en un servidor de producción, ya que ninguna configuración sensible está fijada en el código: credenciales de base de datos y clave JWT se externalizan completamente mediante variables de entorno. A continuación se detallan los pasos precisos para poner en marcha el sistema.

6.1 Requisitos previos

- JDK 17 o superior.
- Apache Maven 3.9+ (o el wrapper mvnw/mvnw.cmd si está presente en el proyecto).
- MySQL 8+ (o MariaDB 10.4+ como alternativa compatible).
- Un cliente HTTP para probar la API REST — Postman, Insomnia o curl (opcional, recomendado para validar /api/auth/login).

6.2 Paso 1 — Obtener el código fuente

```
git clone <URL-DEL-REPOSITORIO>
cd task-manager
```

6.3 Paso 2 — Configurar el servidor de base de datos

Se debe tener una instancia de MySQL 8 en ejecución (local o en un servidor remoto/proveedor cloud) y ejecutar, en orden, los dos scripts SQL incluidos en el proyecto:

```
# 1) Crea la base de datos y las tablas (users, tasks) con sus
#     claves foráneas y restricciones UNIQUE
mysql -u root -p < database/schema.sql

# 2) Inserta las dos cuentas de prueba con contraseñas ya
#     encriptadas en BCrypt
mysql -u root -p < database/data.sql
```

Como alternativa —o como red de seguridad—, si estos scripts no se ejecutan manualmente, el componente DataInitializer sembrará automáticamente las mismas cuentas y un conjunto de tareas de ejemplo la primera vez que la aplicación arranque contra una base de datos vacía. Los scripts SQL, sin embargo, son la vía recomendada para una instalación reproducible y auditable, y son los que se documentan en este informe como scripts de inicio y de poblamiento.

6.4 Paso 3 — Configurar las variables de entorno

Ninguna credencial va escrita en application.properties ni en el código fuente. Antes de ejecutar la aplicación se deben definir las siguientes variables de entorno del sistema operativo o del IDE:

Variable	Valor esperado en el servidor de producción
DB_URL	jdbc:mysql://<host>:3306/taskmanager_db?useSSL=false&serverTimezone=America/Lima&allowPublicKeyRetrieval=true
DB_USERNAME	Usuario de MySQL con permisos sobre taskmanager_db
DB_PASSWORD	Contraseña real del usuario de MySQL (nunca la de desarrollo)
JWT_SECRET	Clave Base64 nueva y aleatoria de al menos 256 bits, exclusiva de producción
JWT_EXPIRATION	Tiempo de vida del token en milisegundos (por defecto 3600000 = 1 hora)

Ejemplo de definición en Windows PowerShell (entorno local del equipo o servidor Windows):

```
$env:DB_USERNAME = "usuario_produccion"
$env:DB_PASSWORD = "contraseña_segura"
$env:JWT_SECRET = "<clave-base64-generada-para-produccion>"
```

Ejemplo equivalente en Linux/macOS (típico servidor de producción):

```
export DB_USERNAME=usuario_produccion
export DB_PASSWORD=contraseña_segura
export JWT_SECRET=<clave-base64-generada-para-produccion>
```

6.5 Paso 4 — Compilar y ejecutar la aplicación

Con Maven instalado y las variables de entorno ya definidas en la misma terminal/sesión:

```
mvn spring-boot:run
```

Este comando descarga las dependencias declaradas en pom.xml, compila el proyecto, ejecuta DataInitializer (si corresponde) y levanta el servidor Tomcat embebido escuchando en el puerto configurado en application.properties. Para generar un artefacto desplegable de forma independiente (por ejemplo, para un servidor remoto sin Maven instalado), se empaqueta primero:

```
mvn clean package
java -jar target/task-manager-1.0.0.jar
```

6.6 Paso 5 — Verificar el despliegue

Una vez arrancado, el sistema queda disponible en <http://localhost:8081> (o en el dominio/IP del servidor de producción, en el puerto configurado). Se recomienda validar el despliegue con las dos cuentas de prueba:

Usuario	Contraseña	Rol
admin	admin123	ADMIN
usuario	user123	USER

Adicionalmente, puede verificarse que la capa de autenticación por API funcione correctamente con una petición directa al endpoint REST:

```
curl -X POST http://localhost:8081/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin123"}'
```

Una respuesta exitosa devuelve un JSON con el token JWT, el nombre de usuario y sus roles, confirmando que la base de datos, la capa de seguridad y el servidor están correctamente enlazados.

6.7 Entorno de ejecución objetivo

El proyecto no depende de ningún servicio propietario de un proveedor específico: es un artefacto Spring Boot estándar (JAR ejecutable con Tomcat embebido) más una base de datos MySQL 8, por lo que puede desplegarse indistintamente en un equipo local del equipo de desarrollo (entorno académico actual del curso), en una máquina virtual o servidor on-premise de PrymEvolution, o en cualquier proveedor cloud compatible con aplicaciones Java/Spring Boot y MySQL administrado, sin cambios en el código — únicamente reconfigurando las cinco variables de entorno descritas en la sección 6.4.